Multimodal autonomous agent application

Multimodal autonomous agent application
- 1. Concept Introduction
  - 1.1 What is an "Autonomous Agent"?
  - 1.2 Implementation Principles
- 2. Code Analysis
  - Key Code
    - 1. Agent Core Workflow (`largemodel/utils/ai_agent.py`)
    - 2. Mission planning and LLM interaction (`largemodel/utils/ai_agent.py`)
    - 3. Parameter processing and data flow implementation (`largemodel/utils/ai_agent.py`)
  - Code Analysis
- 3. Practical Operations
  - 3.1 Configuring the Offline Large Model
    - 3.1.1 Configuring the LLM Platform (`yahboom.yaml`)
    - 3.1.2 Configuration model interface (`large_model_interface.yaml`)
  - 3.2 Starting and Testing the Functionality (Text Input Mode)
- 4. Common Problems and Solutions
  - 4.1 Abnormal Agent Behavior
    - Issue 1: The agent is stuck in an infinite loop or repeatedly executing the same tool.
  - 4.2 Tool Invocation Failure
    - Issue 2: The agent correctly planned the action, but the tool execution failed.

---

1. Concept Introduction

1.1 What is an "Autonomous Agent"?

In the `largemodel` project, **multimodal autonomous agents** represent the most advanced form of intelligence. Rather than simply responding to a user's command, they are capable of **autonomously thinking, planning, and continuously invoking multiple tools to achieve a complex goal**.

The core of this functionality is the **agent_call** tool or its underlying **ToolChainManager**. When a user issues a complex request that cannot be accomplished with a single tool call, the autonomous agent is activated.

1.2 Implementation Principles

The autonomous agent implementation in `largemodel` follows the industry-leading **ReAct (Reason + Act)** paradigm. Its core concept is to mimic the human problem-solving process, cycling between "thinking" and "acting."

1. **Reason**: When the agent receives a complex goal, it first invokes a powerful language model (LLM) to perform "thinking." It asks itself, "To achieve this goal, what should I do first? Which tool should I use?" The output of the LLM isn't a final answer, but rather an action plan.
2. **Act**: Based on the LLM's deliberation, the agent performs the corresponding action—calling the ToolsManager to run the specified tool (e.g., `visual_positioning`).

3. **Observe**: The agent obtains the result of the previous action ("observation"), for example, `{"result": "The cup was found at [120, 300, 180, 360]"}`.
4. **Rethink**: The agent submits this observation, along with the original goal, to the LLM for a second round of "reflection." It asks itself, "I've found the location of the cup. What should I do next to learn its color?" The LLM might generate a new action plan, such as `{"thought": "I need to analyze the image of the area where the cup is located to determine its color", "action": "seewhat", "args": {"crop_area": [120, 300, 180, 360]}}`.

This **think -> act -> observe** cycle continues until the initial goal is achieved, at which point the agent generates and outputs the final answer.

2. Code Analysis

Key Code

1. Agent Core Workflow (`largemodel/utils/ai_agent.py`)

The `_execute_agent_workflow` function is the agent's main execution loop, defining the core "plan -> execute" process.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...

    def _execute_agent_workflow(self, task_description: str) -> Dict[str, Any]:
        """
        Executes the agent workflow: Plan -> Execute. / 执行Agent工作流：规划 -> 执行。
        """
        try:
            # Step 1: Mission Planning
            self.node.get_logger().info("AI Agent starting task planning phase")
            plan_result = self._plan_task(task_description)

            # ... (Return early if planning fails)

            self.task_steps = plan_result["steps"]

            # Step 2: Follow all steps in order
            execution_results = []
            tool_outputs = []

            for i, step in enumerate(self.task_steps):
                # 2.1. Processing data references in parameters before execution
                processed_parameters = self._process_step_parameters(step.get("parameters", {}), tool_outputs)
                step["parameters"] = processed_parameters

                # 2.2. Executing a single step
                step_result = self._execute_step(step, tool_outputs)
                execution_results.append(step_result)

                # 2.3. If the step succeeds, save its output for reference in subsequent steps
```

```
                if step_result.get("success") and
step_result.get("tool_output"):
                    tool_outputs.append(step_result["tool_output"])
                else:
                    # If any step fails, abort the entire task
                    return { "success": False, "message": f"Task terminated
because step '{step['description']}' failed." }

            # ... Summarize and return the final result
            summary = self._summarize_execution(task_description,
execution_results)
            return { "success": True, "message": summary, "results":
execution_results }

        # ... (Exception handling)
```

2. Mission planning and LLM interaction (`largemodel/utils/ai_agent.py`)

The core of the `_plan_task` function is to build a sophisticated prompt and use the reasoning ability of the large model to generate a structured execution plan.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _plan_task(self, task_description: str) -> Dict[str, Any]:
        """
        Uses the large model for task planning and decomposition. / 使用大模型进行
任务规划和分解。
        """
        # Dynamically generate a list of available tools and their descriptions
        tool_descriptions = []
        for name, adapter in
self.tools_manager.tool_chain_manager.tools.items():
            # ... (Get tool description from adapter.input_schema)
            tool_descriptions.append(f"- {name}({params}): {description}")
        available_tools_str = "\\n".join(tool_descriptions)

        # Build a highly structured plan
        planning_prompt = f"""
As a professional task planning Agent, please break down the user's task into a
series of specific, executable JSON steps.

**# Available Tools:**
{available_tools_str}

**# Core Rules:**
1.  **Data Passing**: When a subsequent step needs to use the output of a
previous step, you **must** use the `{{{{steps.N.outputs.KEY}}}}` format for
referencing.
    - `N` is the step ID (starting from 1).
    - `KEY` is the specific field name in the output data of the previous step.
    - `outputs` can be followed by `data` (for primary data) or
`metadata.sub_key` (for metadata).
2.  **JSON Format**: You must strictly return a JSON object. Do not include any
Markdown wrappers (like ```json```).
```

```
3.  **Tool Selection**: Strictly select the most appropriate tool based on its
description.

**#  User Task:**
{task_description}
"""

        # Calling large models for planning
        messages_to_use = [{"role": "user", "content": planning_prompt}]
        # Note that the general text reasoning interface is called here
        result = self.node.model_client.infer_with_text("",
message=messages_to_use)

        # ... (Parse the JSON response and return a list of steps)
```

3. Parameter processing and data flow implementation (`largemodel/utils/ai_agent.py`)

The `_process_step_parameters` function is responsible for parsing placeholders and implementing data flow between steps.

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _process_step_parameters(self, parameters: Dict[str, Any],
previous_outputs: List[Any]) -> Dict[str, Any]:
        """
        Parses parameter dictionary, finds and replaces all {{...}} references.
        """
        processed_params = parameters.copy()
        # Regular expression used to match placeholders in the format
{{steps.N.outputs.KEY}}
        pattern = re.compile(r"\\{\\{steps\\.(\\d+)\\.outputs\\.(.+?)\\}\\}")

        for key, value in processed_params.items():
            if isinstance(value, str) and pattern.search(value):
                # Use re.sub and a replacement function to process all found
placeholders.
                # The replacement function will look up and return the value from
the previous_outputs list.
                processed_params[key] = pattern.sub(replacer_function, value)

        return processed_params
```

Code Analysis

The AI Agent is the "brain" of the system, translating high-level, sometimes ambiguous, tasks posed by the user into a precise, ordered series of tool calls. Its implementation is independent of any specific model platform and built on a general, extensible architecture.

1. **Dynamic Task Planning**: The Agent's core capability lies in the `_plan_task` function. Rather than relying on hard-coded logic, it dynamically generates task plans by interacting with a larger model.

- **Self-Awareness and Prompt Construction**: At the beginning of planning, the Agent first examines all available tools and their descriptions. It then packages this tool information, the user's task, and strict rules (such as data transfer format) into a highly structured `planning_prompt`.
- **Model as Planner**: This prompt is fed into a general text-based model. The model reasoned based on the provided context and returned a multi-step action plan in JSON format. This design is highly scalable: as tools are added or modified in the system, the Agent's planning capabilities are automatically updated without requiring code modifications.

2. **Toolchain and Data Flow**: Real-world tasks often require the collaboration of multiple tools. For example, "take a picture and describe" requires the output (image path) of the "take a picture" tool to be used as the input of the "describe" tool. The AI Agent elegantly implements this through the `_process_step_parameters` function.

- **Data Reference Placeholders**: During the planning phase, large models embed special placeholders, such as `{{steps.1.outputs.data}}`, in parameter values where data needs to be passed.
- **Real-Time Parameter Replacement**: In the `_execute_agent_workflow` main loop, `_process_step_parameters` is called before each step. It uses regular expressions to scan all parameters of the current step. Upon discovering a placeholder, it finds the corresponding data from the output list of the previous step and replaces it in real time. This mechanism is key to automating complex tasks.

3. **Supervised Execution and Fault Tolerance**: `_execute_agent_workflow` constitutes the Agent's main execution loop. It strictly follows the planned sequence of steps, executing each action sequentially and ensuring data is correctly passed between them.

- **Atomic Steps**: Each step is treated as an independent "atomic operation." If any step fails, the entire task chain immediately aborts and reports an error. This ensures system stability and predictability, preventing continued execution in an erroneous state.

In summary, the general implementation of the AI Agent demonstrates an advanced software architecture: rather than solving a problem directly, it builds a framework that enables an external, general-purpose reasoning engine (a large model) to solve the problem. Through two core mechanisms, dynamic programming and data flow management, the Agent orchestrates a series of independent tools into complex workflows capable of completing advanced tasks.

3. Practical Operations

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. **Open the file in the terminal**:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. **Modify/confirm `llm_platform`**:

```
model_service:                          #模型服务器节点参数 Model server node parameters
  ros__parameters:
    language: 'en'                      #大模型接口语言 Large Model Interface Language
    useolinetts: False                   #文字模式下此项无效，可忽略 This item is invalid in text mode and can be ignored

    # 大模型配置 Large model configuration
    llm_platform: 'ollama'              # 关键: 确保这里是 'ollama' Key: Make sure it's 'ollama'
    regional_setting : "international"
```

3.1.2 Configuration model interface (`large_model_interface.yaml`)

This file defines which visual model to use when the platform is selected as `ollama`.

1. Open the file in a terminal.

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the configuration related to ollama

```
#.....
## 离线大模型 (Offline Large Language Models)
# Ollama Configuration
ollama_host: "http://localhost:11434"  # Ollama server address
ollama_model: "llava"            # Key: Change this to the multimodal model you downloaded, such as "llava"
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

If you have modified any configuration files, please run the following command to compile the ROS workspace for the configuration to take effect.

```
cd ~/yahboom_ws
colcon build
```

3.2 Starting and Testing the Functionality (Text Input Mode)

Note: Due to performance limitations, the performance of the Jetson Orin Nano 4GB is limited. To experience this feature, please refer to the "Online Large Model (Text Interaction)" section.

1. **Start the `largemodel` main program (text mode):**
   Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

2. **Send text command**:
   Open another terminal and run the following command:

   ```
   ros2 run text_chat text_chat
   ```

   Then start entering text: "Based on the current environment, save the generated environment description as a txt document."

3. **Observe the Results**:

In the first terminal running the main program, you will see log output indicating that the system receives the text command, invokes the `aiagent` tool, and then provides a prompt to LLM. LLM will analyze the detailed steps of the tool invocation. For example, in this question, the `seewhat` tool will be invoked to capture the image, which will then be parsed by LLM. The parsed text will be saved in the `~/yahboom_ws/src/largemodel/resources_file/documents` folder.

---

4. Common Problems and Solutions

4.1 Abnormal Agent Behavior

Issue 1: The agent is stuck in an infinite loop or repeatedly executing the same tool.

Solution:

1. **Enhance Prompt**: Add stronger restrictions to the ReAct prompt, such as "Do not perform the same action repeatedly" or "If two consecutive observations show the same result, try a different tool or end the task."
2. **Increase Iteration Limit**: For example, `max_turns` in the code example, set a maximum number of iterations to prevent infinite loops.
3. **Replace a More Powerful LLM**: The agent's logical capabilities largely depend on the intelligence of the backend LLM. A more powerful model can better understand the task and make planning decisions.

4.2 Tool Invocation Failure

Issue 2: The agent correctly planned the action, but the tool execution failed.

Solution:

1. **Check individual tools**: This is usually not a problem with the proxy logic, but rather with the called tool (such as `seewhat` or `visual_positioning`). Please follow the corresponding tool's instructions to troubleshoot the issue.
2. **Error Handling**: Within the `ToolChainManager` loop, a comprehensive `try...except` mechanism should be implemented to catch tool execution exceptions and report the error information as an "observation" to the LLM, letting it know that the previous step failed and allowing it to re-plan.